

Music Playback III

Week 11.



COMT

SOFTWARE CLUSTER

// COMT • **COMPUTATIONAL THINKING** (CIT1C18)



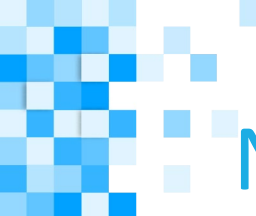
Overview

- In this workshop, you will be learning how to play the next/previous song and also the stopping of the music when user traverse back to the main screen.



Objectives

- Demonstrate how to use Android **MediaPlayer**;
- Demonstrate how to write conditional statements;
- Demonstrate how to write methods and parameters.
- Demonstrate how to traverse Array.



Note

- Throughout this document, the acronym “**app**” will be used to refer to **mobile application**.

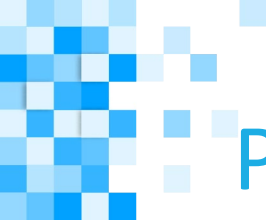


PLAYING THE NEXT SONG

Playing Next Song

- In this lesson, you will be learning how to code and play the next song when the **NEXT** button is clicked.
- So how does the player know which is the next song in the list to be played?





Playing Next Song

- For the system to know which is the next song to be played, there are many different ways to implement such logic
- We will create a new method for this task and name it `getNextSong` in the `SongCollection` java class

Playing Next Song

- The codes for `getNextSong` method is shown below.
- Add this code in **SongCollection.java**.

```
public int getNextSong(int currentSongIndex){  
    if (currentSongIndex >= songs.length-1){  
        return currentSongIndex;  
    }  
    else{  
        return currentSongIndex +1;  
    }  
}
```

The method will accept an integer as the argument. Assuming the value is correct, the codes will check if this value is bigger or equal to the number of items in the array.

If it is greater or equal, then it means that it is currently at the last song and there will **NOT** be a next song. Therefore it will return back the current index.

If it is smaller than the number of songs in the array, it means that there **IS** a next song and therefore it will return the value + 1, which is the position of the next song

Playing Next Song

- Lets try to use the `getNextSong` method
- Add this method `playNext` in **PlaySongActivity.java**.

Your `btnNext` button should already be wired to call this method when it is being tapped

The variable `currentIndex` is a class variable and it holds the array position of the current selected song.
Here we call the `getNextSong` and pass in this variable `currentIndex` because it represents the position of song in the array

Toast out information for us to know the array position

```
public void playNext(View view) {  
    currentIndex = songCollection.getNextSong(currentIndex);  
    Toast.makeText(context: this, text: "After clicking playNext, \nthe current index of this song\n" +  
        "in the SongCollection array is now :" + currentIndex, Toast.LENGTH_LONG).show();  
    Log.d(tag: "temasek", msg: "After playNext, the index is now :" + currentIndex);  
    displaySongBasedOnIndex(currentIndex);  
    playSong(fileLink);  
}
```

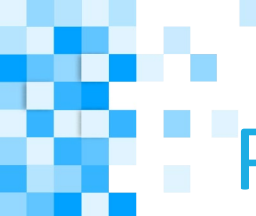
Update the ui based on the new `currentIndex` value

Start playing the song

Playing The Previous Song

- Implementing the play previous song is very similar in idea to the play next song method.





Playing The Previous Song

- For the system to know which is the previous song to be played, there are many different ways to implement such logic
- We will create a new method for this task and name it `getPrevSong` in the `SongCollection` java class

Playing The Previous Song

- The codes for `getPrevSong` method is shown below.
- Add this method to **SongCollection.java**.

```
public int getPrevSong(int currentSongIndex){  
    if (currentSongIndex <= 0){  
        return currentSongIndex;  
    }  
    else{  
        return currentSongIndex-1;  
    }  
}
```

The method will accept an integer as the argument. Assuming the value is correct, it will check if this value is lesser or equal to zero

If it is lesser or equal to zero, it means that it is currently the first song and therefore it will just return back itself

If it is greater than zero, it means that there is a previous song and therefore it will return the value - 1, which represent the position of this song

Playing The Previous Song

- Lets try to use the `getPrevSong` method
- Add this method in **PlaySongAcitivity.java**.

Your `btnPreviousbutton` should already be wired to call this method when it is being tapped

The variable `currentIndex` is a class variable and it holds the array position of the current selected song.
Here we call the `getPrevSong` and pass in this variable `currentIndex`

Toast out information for us to know the array position

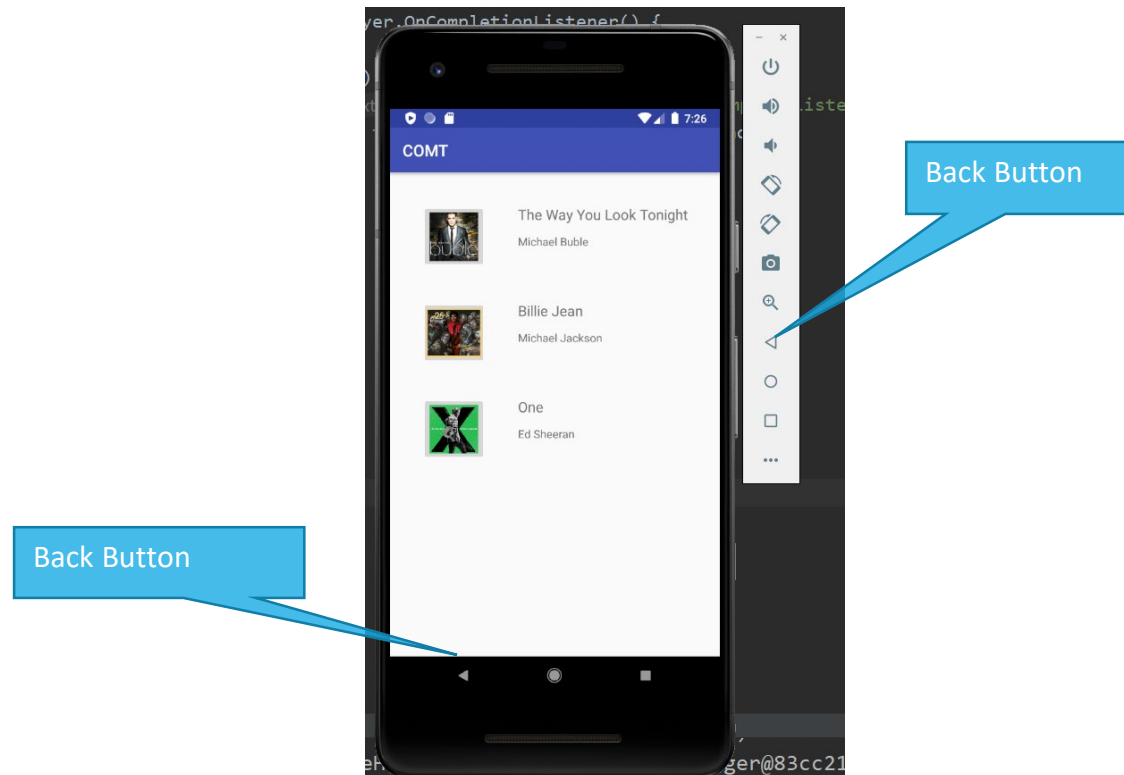
```
public void playPrevious(View view) {  
    currentIndex = songCollection.getPrevSong(currentIndex);  
    Toast.makeText(context: this, text: "After clicking playPrevious,\nthe current index of this song\n" +  
        "in the SongCollection array is now :" + currentIndex, Toast.LENGTH_LONG).show();  
    Log.d(tag: "temasek", msg: "After playPrevious, the index is now :" + currentIndex);  
    displaySongBasedOnIndex(currentIndex);  
    playSong(fileLink);  
}
```

Update the ui based on the new `currentIndex` value

Start playing the song

Stopping the Song

- At this moment, whenever you traverse back (by pressing the back button), the song continues to play and will go on to play an additional track if you tapped on the 2nd song. (This is a bug)
- We will create a new method so that the song is stopped whenever the user traverse back to the MainActivity



Stopping the Song

- The codes for `onBackPressed` method is shown below.
- Add this code in **PlaySongActivity.java**.

```
@Override
public void onBackPressed() {
    super.onBackPressed();
    player.release();
}
```

This method will be executed whenever the app sense that the “back” button is being tapped.

Here we run the code `player.release()` to stop the music and also release the resources used by the `mediaPlayer`.



Summary

- Congratulations! You have created a basic Android streaming music player!
- Practise by adding more songs to the ui and to the songs array



Supplementary Materials

Topic	Source
Java Method/Function	https://www.learnjavaonline.org/en/Functions
	https://mathbits.com/MathBits/Java/Methods/Lesson1.htm
	http://www.horsesnw.com/articles/JavaMethod.htm
Java Conditionals	https://www.learnjavaonline.org/en/Conditionals
Java break	https://www.tutorialspoint.com/java/java_break_statement.htm
Android MediaPlayer	https://developer.android.com/guide/topics/media/mediaplayer.html
	http://www.androidhive.info/2012/03/android-building-audio-player-tutorial/
	https://www.tutorialspoint.com/android/android_mediaplayer.htm

THE END



COMT